

CSE 587 - Data Intensive Computing Project

Phase-3

Name	UBIT Name	UB Person Number
Rishika Reddy Thumma	rthumma	50560552
Mahidhar Reddy Indela	mindela	50559954

Code Documentation:

- **Phase 1:**

1. Data is initially loaded from our dataset.
2. Dropping unnecessary columns and missing values from dependent variables.
3. Handle missing values in categorical data.
4. Convert data types to ensure consistency across datasets.
5. Case standardization and cleaning the text columns to enhance code readability and accessibility.
6. Encode categorical data.
7. Filter out irrelevant data and outliers
8. We have plotted histograms, scatter plots and box plots for univariate, bivariate and multivariate analysis of the data.
9. We have plotted correlation matrix to assess the relationship among the features.

- **Phase 2:**

We included the data cleaning code from Phase 1 in the initial part of Phase 2 to ensure continuity and integrity of data processing.

Models used in our project:

- **Linear Regression:**

Linear regression is used to model the relationship between game ratings and detected technologies, assuming a direct linear impact of technologies on ratings.

Model Evaluation: The model shows moderate effectiveness with an R-squared value of approximately 0.615, indicating it captures around 61.48% of the variance in game ratings. However, high MAE, MSE, and RMSE values suggest the presence of significant prediction errors possibly due to outliers or unmodeled variables.

- **KNN Algorithm:**

KNN is used for its versatility in handling non-linear data relationships and its ability to perform both classification and regression tasks without assuming an underlying data distribution.

Model Evaluation: The model demonstrates a decent fit, with an R-squared value of 0.882. However, the variability in prediction accuracy, especially for higher ratings, suggests room for improvement through parameter optimization or feature scaling.

- **Decision Tree:**

Decision trees are employed to delineate complex relationships in the data by splitting based on feature values, useful for identifying critical technological features affecting player engagement.

Model Evaluation: The decision tree model shows a high effectiveness, with an R-squared value near 0.983, indicating it accounts for a substantial portion of the variance in the

data. The model is noted for its precision and robust fit to the dataset.

- **Random Forest:**

Random forests are used to enhance prediction accuracy and manage overfitting by averaging multiple decision trees trained on different data subsets. This model is particularly effective due to its robustness and complexity handling capability.

Model Evaluation: The model performs exceptionally well, with an R-squared value close to 0.991, suggesting it nearly perfectly captures the variance in the data. The model is highly accurate, as evidenced by low MAE and RMSE values.

- **Gradient Boosting:**

Gradient Boosting is suitable for handling complex and non-linear correlations in data involving numerous numerical parameters that influence game ratings.

Model Evaluation: The model is highly effective, with an R-squared value of nearly 0.994, indicating a very strong fit to the data. It offers precise predictions with a low MAE and RMSE, demonstrating its capability to handle the dataset effectively.

- **XGBoost:**

XGBoost is chosen for its efficiency and flexibility, capable of handling complex and nonlinear relationships in data. It's optimized for performance and scalability, making it suitable for large datasets.

Model Evaluation: The model shows strong predictive accuracy with an R-squared value close to 0.979. It demonstrates robust performance across various metrics, indicating its effective fit to the data and high explanatory power.

- **Phase 3:**

- We have used flask to run our website code in `http://127.0.0.1:5000`

- Each function contains flask code for each element in the web application. The backend connection for our model is established properly.

- We identified top game technologies that can be used to predict the rating of the game engine and sdk's that are used to develop a game.

- And we end up using gradient boosting as it is effective in capturing complex non-linear relationships between numerical features and the ratings i.e target variable. And the evaluation of the Gradient Boosting model reveals an impressive R-squared value of nearly 0.994, indicating an exceptionally strong fit to the data. This means that the model captures almost all the variance in the game ratings, suggesting its ability to provide accurate predictions.

- The system provides the rating of game engines based on the player reviews and number of players who plays the game.

User Interface working instructions:

Use the below commands to run our game tech rating prediction website in flask web user interface.

- Install flask python module using the following command: `pip3 install flask`
- And then run our application code as python module with the following command: `/Users/mahidharreddyindela/Documents/dicphase3/app.py`
- Here the user have to give the following inputs to predict the rating of the game for a game technology used to develop the game.

- Peak Players: Enter the number of peak players.
- Positive Reviews: Input the total number of positive reviews.
- Negative Reviews: Enter the total number of negative reviews.
- Total Reviews: Input the total number of reviews.
- Review Percentage: Enter the percentage of positive reviews.
- Current Players: Input the current number of players online.
- Hour Peak (24 Hour): Enter the peak hourly player count in 24 hr.
- All Time Peak: Input the all-time peak player count.
- Select Engine and SDKs: Choose an appropriate game engine and SDKs from the dropdown menu labeled "Select Engine and SDKs."
- And click the "Submit" button to process the entered data. The form data is sent to the server, where it is presumably used to make a prediction based on the model.
- After submission, if a prediction is made by the backend and passed back to the frontend, it will be displayed under the form. There is a conditional block in the HTML that checks for a prediction variable and, if present, displays it.

Analysis and future recommendations:

For our project we used gradient boosting to predict outcomes based on the game technology data from the dataset. This method is particularly suitable for handling complex datasets with multiple features influencing the outcome, such as the game ratings and player engagement metrics.

Our system can function as a valuable aid for game developers, offering guidance on how various game technologies, such as game engines and SDKs, influence player feedback and game ratings. By providing this information, developers can make well-informed decisions about the technologies they employ in their game designs.

Our system can also be used in optimizing game development processes by anticipating player reactions through technology selection, developers can streamline their game development processes, allocating resources towards technologies that are more inclined to improve player satisfaction and involvement. and also our system assists developers in selecting technologies that have a proven track record of boosting player satisfaction, thereby enhancing the quality of games and improving the overall player experience.

We can further develop the project enhance predictive models by experimenting with different machine learning algorithms. Considering neural networks to handle the complex relationships and heterogeneity in your data. Also, evaluate models not just on accuracy but also on metrics like F1-score or AUC, particularly when dealing with imbalanced datasets. And integrating additional datasets that may include more direct measurements of player experience, such as user interaction data from gameplay sessions or social media sentiment analysis. This could enrich your models and provide a more nuanced understanding of player satisfaction.

Create a data processing pipeline that enables the model to generate real-time insights, offering instant feedback. This feature is especially beneficial for live games, allowing developers to dynamically adjust game features based on immediate feedback to enhance the player experience.

We can further enhance analytics capabilities by implementing advanced techniques like feature importance analysis, which identifies the variables with the greatest influence on player engagement and ratings. This insight can inform more targeted development approaches within the gaming sector. Additionally, employ model explainability tools such as SHAP (SHapley Additive exPlanations) to

interpret predictions made by the Gradient Boosting model. This transparency is essential for stakeholders to comprehend and trust the model's decisions.

Data Product deployment:

Explore Game Tech : Ratings

Peak Players:	Positive Reviews:
Negative Reviews:	Total Reviews:
Review Percentage:	Current Players:
Hour Peak (24 Hour):	All Time Peak:
Select Engine and SDKs:	
Engine:gamemaker	Submit Reset

Explore Game Tech : Ratings

Peak Players:	Positive Reviews:
2484	4594
Negative Reviews:	Total Reviews:
234	4828
Review Percentage:	Current Players:
96	1245
Hour Peak (24 Hour):	All Time Peak:
2354	4674
Select Engine and SDKs:	
Engine:unity Sdk:oculusxrplugin Sdk:openvr	Submit Reset

Explore Game Tech : Ratings

Peak Players:	Positive Reviews:
Negative Reviews:	Total Reviews:
Review Percentage:	Current Players:
Hour Peak (24 Hour):	All Time Peak:
Select Engine and SDKs:	
Engine:gamemaker	Submit Reset

Rating: [92.17450612]

The above images show web UI of our rating prediction system for our machine learning model: gradient boosting. Each section of our Web UI takes the inputs of different features like peak players , positive reviews , negative reviews , total reviews , review percentage , current players , players in 24 hours , all time peak , and finally game engine and sdk's from the drop down box .

After submitting all the data given by the user it will give the rating for the particular game engine which helps to understand the player reception of that particular game engine and helps us to improve the effectivity of game engine and sdk's used in it.

References:

1. <https://flask.palletsprojects.com/en/3.0.x/>
2. <https://getbootstrap.com/>
3. <https://www.kaggle.com/datasets/whigmawhim/steam-releases>
4. J. VanderPlas, Python Data Science Handbook., O'Reilly. 2016
https://search.lib.buffalo.edu/permalink/01SUNY_BUF/n53bqg/alma9939137484104803